

Choosing Analysis for Time Complexity

Let's consider the search problem we mentioned on the last page. The element we are searching for might be in the first position, the last position, or somewhere in between.

As we don't know where our element might be, we normally consider **the worst-case** analysis for calculating time complexity.



Important: Average case analysis is not always possible for all the algorithms. And most of the time, it is similar to the worst-case analysis.

Was this helpful?

Yes 

No 

Worst case

Suppose we have to search **19** from the `numbers` list.

Then, it would take us **5** steps to search **19**. It's because the list doesn't contain **19**, and we have to search until the end of the list to check if **19** is present in the list or not.

This is the **worst-case** analysis because we have to go through every element present in the list.

Average case

We might need to search for an element that is somewhere in between the first and last elements.

If the element is present in the first position, it would take **1** search. If the element is present in the third position, it would take **3** searches.

Similarly, if the element is present in the `n`th index, it would take `n` searches. This is the **average case** analysis.

$$\text{Average Case} = \frac{\text{sum of all possible cases}}{n}$$

Therefore, the average case analysis to search for an element in the `numbers` list is:

$$\text{Average Case} = \frac{1 + 2 + 3 + 4 + 5}{5} = 3$$

Was this helpful?

Yes 

No 

Quiz:

Step Count

How many units of time does this function take to run if $n = 3$?

```
def print_num(n):  
    if n > 1:  
        print(n)
```

A. $n+1$

B. $2n$

C. 1

D. 2

Correct Answer!



The `if` condition evaluates to `True`, and both the `if` and `print()` statements each take a single unit of time to run. Thus, the total unit of time taken is 2.

Was this helpful?

Yes

No

Quiz:

Step Count

How many units of time does this function take to run if $n = 3$?

```
def print_num(n):  
    if n > 1:  
        print(n)
```

A. $n+1$

B. $2n$

C. 1

D. 2

Was this helpful?

Yes 

No 

Frequency Count Method

We use the frequency count method to calculate the time complexity of an algorithm. The method is based on the number of steps our program takes to execute (like we did in our earlier example).

Here's the same code to find the sum of the first `n` natural numbers.

```
1 def find_sum(n):
2     # initialize total to zero
3     total = 0
4     # iterate n times
5     for i in range(1, n + 1):
6         # add the number to total
7         total = total + i
8
9     return total
10
11 result = find_sum(10)
12 print(result)
```

Run Code >>

Output

55

Now let's calculate the time complexity of the `find_sum()` function using the frequency count method.

In this method, we assign 1 unit of time to each statement. For example,

```
def find_sum(n):  
    total = 0  
    for i in range(1, n + 1):  
        total = total + i  
    return total
```

unit of time

$= 1 + n + n + 1$
 $= 2n + 2$ units of time

Figure: Frequency count method

If we write the number of steps in terms of a polynomial function, it would be:

$$f(n) = 2n + 2$$

This means the above program takes $2n + 2$ steps to execute.

Now that we know how to calculate the number of steps required to run a program, we will learn about the degree of a polynomial function next.

Understanding the degree of a polynomial will help us to find the time complexity of a given function, `f(n)`.

Basic Time Complexity Example

Now, let's get back to the code we previously wrote.

```
1 total = 0
2 n = 1000
3
4 # iterate n times
5 for i in range(1, n + 1):
6     total = total + i
7
8 print(total)
```

Run Code >>

Output

500500

In this program, if the value of `n` is:

- **5** - the code will take **5** steps to run
- **2000** - the code will take **2000** steps to run

In short, our code takes `n` steps to run.

This calculation is independent of the device or programming language we are using.

Was this helpful?

Yes 

No 

Quiz:

Steps for Execution

How many steps does it take for the following Python code to run?

```
def print_num(n):  
    print(n)  
    print(n+1)
```

A.

B. 1

C. 2



Correct Answer!

As there are two `print()` statements in the code and no loops, the code takes two steps to run.

D.

Introduction

Time complexity is the amount of time an algorithm takes to run (with respect to the size of the input). It is the most important factor to consider while analyzing an algorithm.

But how do we calculate the time complexity of an algorithm?

We could calculate the time complexity of an algorithm based on the execution time. For example, if a code takes **16** minutes to run, its time complexity is **16** minutes.

However, this approach would be wrong because the execution time will be different on different

- devices
- programming languages
- networks
- and so on

That's why we calculate the time complexity based on the number of steps required for a given input. For example,

If the time complexity is **$O(n)$**

- it takes **5** steps for an input of **5**
- it takes **10** steps for an input of **10**

We will cover what **$O(n)$** means in detail later in this course.

For now, just remember that the time complexity is calculated based on the number of steps required (regardless of the device or programming language being used).

Optimizing Code

Now, let's try to optimize the code we wrote earlier.

There is a simple formula to calculate the sum of natural numbers.

$$\text{Sum} = n * (n + 1) / 2$$

Let's code the sum of `n` natural numbers again using the formula.

```
1 n = 1000
2
3 total = n * (n + 1) / 2
4 print(total)
```

Run Code >>

Output

500500.0

Assuming it takes 1 second to run 1 step, our optimized code executes in 1 second.

Our previous code correctly found the sum of natural numbers using a `for` loop, but it took 16 minutes to run.

On the other hand, our optimized code takes only 1 second to run. This is why it is important to calculate the time taken by algorithms.

Even if we want to find the sum of 10^{11} natural numbers, running our optimized code would take the same amount of time as it would for a sum of 1000 natural numbers.

This makes our code **faster** and more **scalable**.

How to Analyze Algorithms?

Analyzing an algorithm depends on various factors, but two factors are particularly important.

- **Time Complexity** - The amount of time taken by an algorithm to run with respect to the size of the input.
- **Space Complexity** - The amount of space that the algorithm takes to run with respect to the size of the input.

In the next chapter, we will focus on **time complexity**.

Was this helpful?

Yes 

No 

Analyzing Algorithms

Suppose we are trying to solve a simple problem of finding the sum of the first **1000** natural numbers. Let's see what our code looks like:

```
1 n = 1000
2 total = 0
3
4
5 # iterate n times
6 for i in range(1, n + 1):
7     total = total + i
8
9 print(total)
```

Run Code >>

Output

500500

This code takes n steps to complete

Suppose our computer executes **one** step in **one** second. In this case, our computer will take $1 * 1000$ seconds, or **16 minutes**, to run **1000** steps. Nobody wants to wait **16** minutes to run a simple code.

Now, imagine if we want the sum of 10^{11} natural numbers. It will take $1 * 10^{11}$ seconds, which is **1666666666 min 40 seconds**.

That is why optimizing algorithms is important so our code is more efficient.

Why Analyze Algorithms?

Suppose you need to travel from New York to California. You can reach California by car, bus, train, or flight.

Flying is the fastest way to get to California, but it is also more expensive compared to taking the train.

Therefore, flying is a good option if you want to get to California quickly. However, taking the train is a better choice if you want a cheap mode of transportation.

Similarly, in computer programming, there are multiple ways to solve the same problem. Analyzing algorithms helps us determine the most efficient way of solving a problem.

Was this helpful?

Yes 

No 

Algorithms

An algorithm is a well-defined set of instructions to solve a specific problem.

For example, an algorithm to print the sum of two numbers might look like this:

Problem: Add Two Numbers

```
Step1: Initialize two variables number1 and number2.  
Step2: Add number1 and number2.  
Step3: Print the total.
```

In this example, the algorithm is written in simple language that anyone can understand. However, if we use a programming language to write it, it is called **code**.

Let's see the Python code to add two numbers.

```
1 # initialize two variables  
2 number1 = 33  
3 number2 = 2  
4  
5 # add the numbers  
6 total = number1 + number2  
7  
8 # print total  
9 print(total)
```

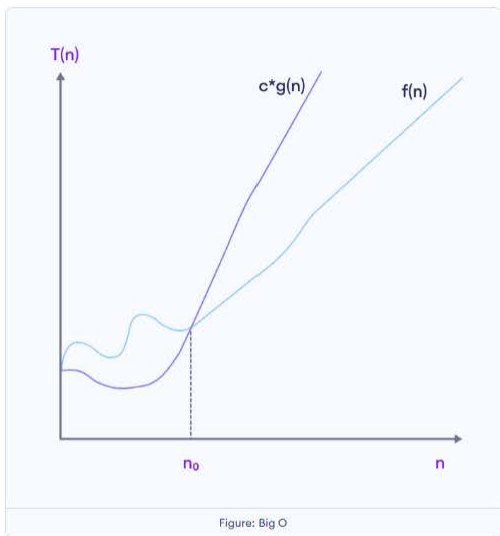
Run Code >>

Output

35

Big O

Here's the same graph of the $f(n)$ function and the $c * g(n)$ we created on the last page.



In this graph, suppose these conditions are met.

1. $f(n) \leq c * g(n)$

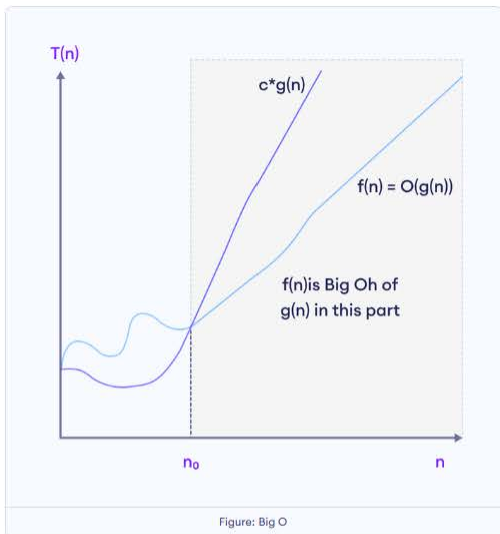
2. $n_0 > 1$ and $n \geq n_0$

3. $c > 0$

Then, we can say that:

$$f(n) = O(g(n)) \text{ (read as } f(n) \text{ is Big Oh of } g(n))$$

Let's visualize where the above conditions are met in the above graph.



Here, the growth of $c * g(n)$ is always faster than $O(g(n))$ on the highlighted part of the image.

Hence, $c * g(n)$ is called the **upper bound** because for all values of n in the highlighted part, the value of $c * g(n)$ is always equal to or greater than $O(g(n))$.

Lingo: $f(n)$ grows no faster than $g(n)$.

Next, we will relate this mathematical interpretation to time complexity.

Was this helpful?

Yes

No

Here,

- c is a positive constant
- the function $f(n)$ will always be equal to or below the function $c \cdot g(n)$ after the n_0 point

In this case, $c \cdot g(n)$ is called the **upper bound** of $f(n)$ because the $f(n)$ function will never grow faster than $c \cdot g(n)$.

But what's the point of learning this mathematical interpretation? After all, we are learning computer science here.

We will return to this topic later in this lesson and connect the mathematical interpretation of the Big O notation in terms of time complexity.

For now, let's complete the mathematical interpretation first.

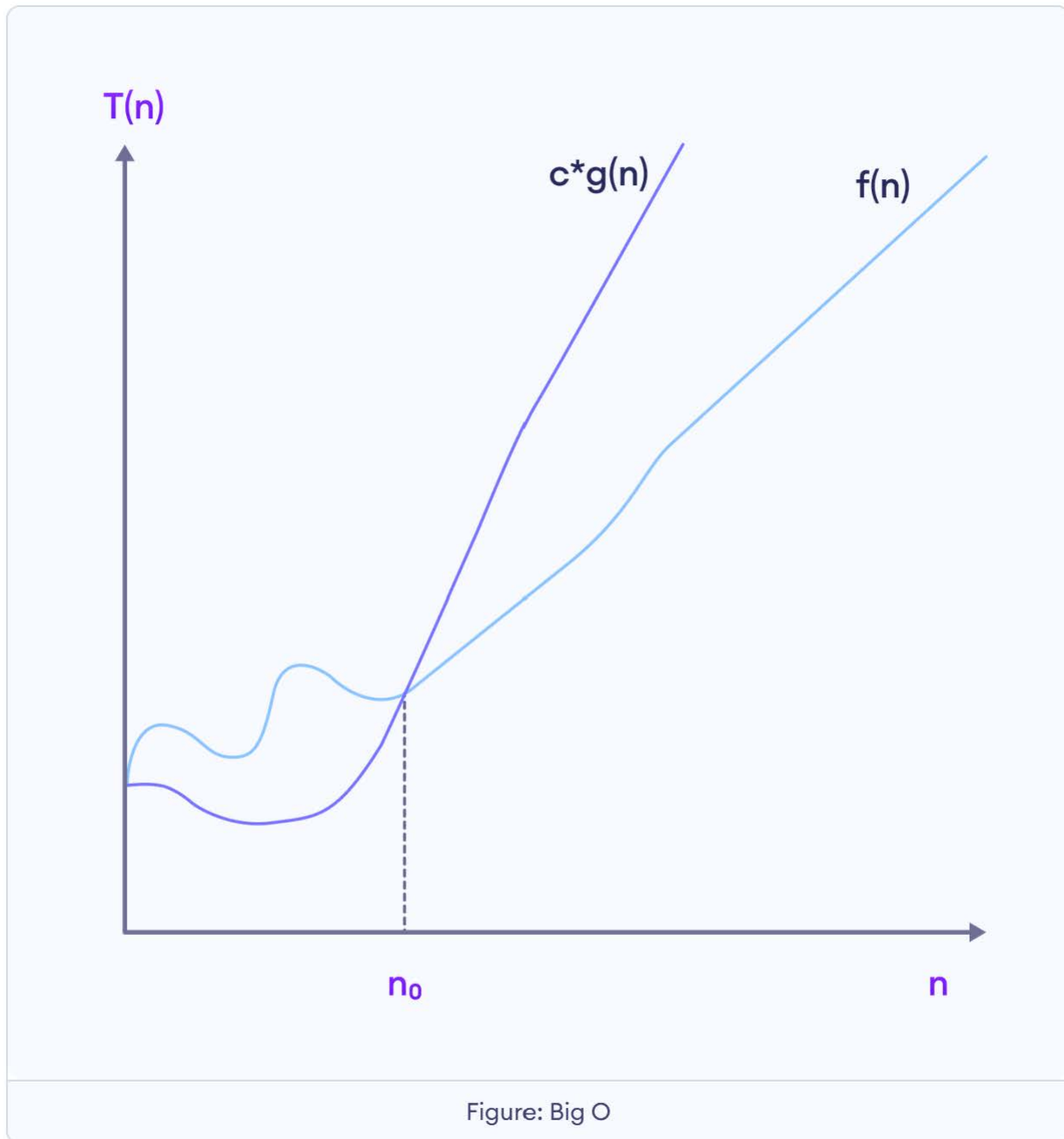
Was this helpful?

Yes 

No 

Here, $f(n)$ is the graph of our function.

Now, we will plot another function in the graph, $c * g(n)$ that will intersect $f(n)$ at the n_0 point.



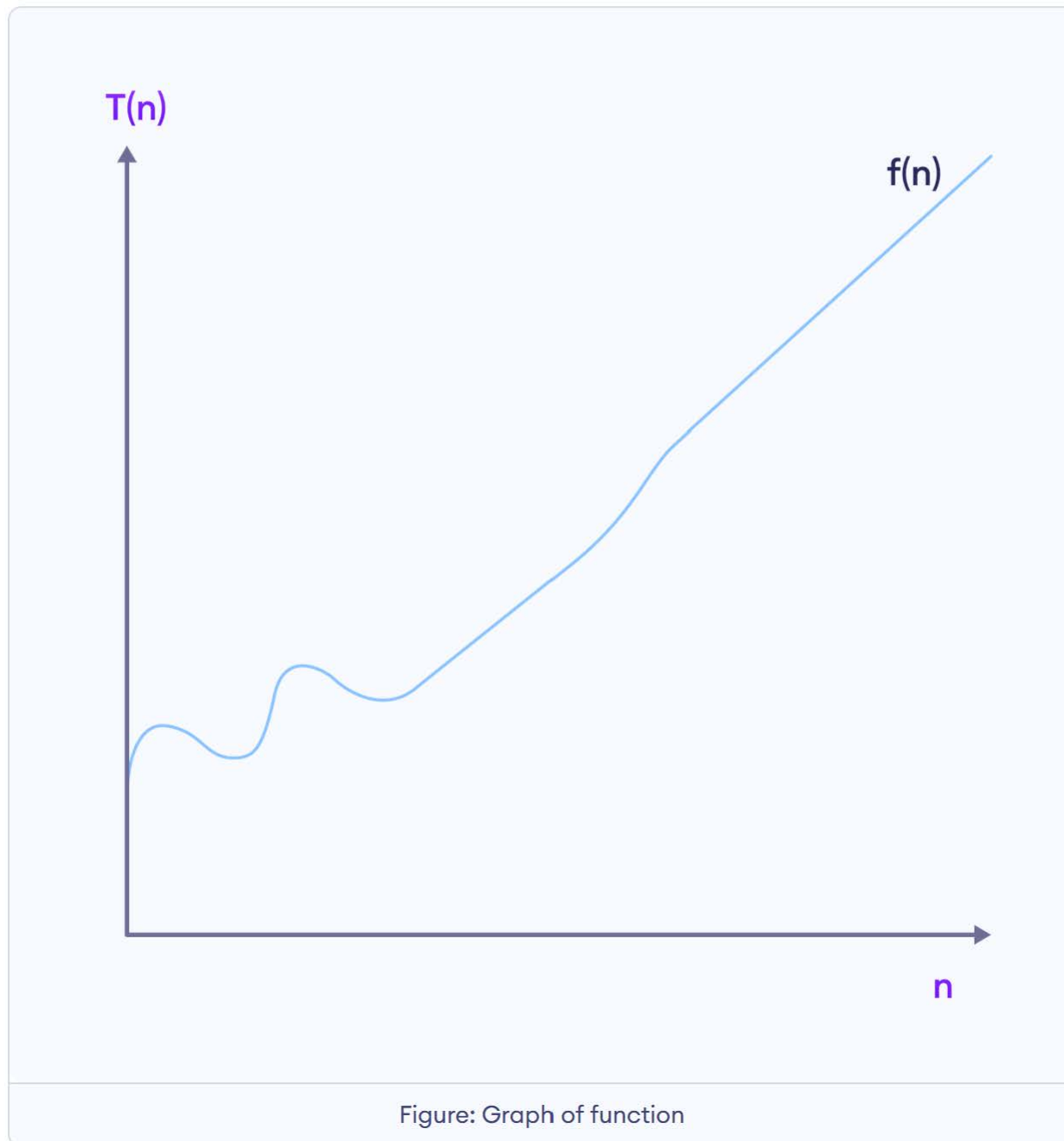
Here,

- c is a positive constant
- the function $f(n)$ will always be equal to or below the function $c * g(n)$ after the n_0 point

Big O Interpretation

The Big O notation represents the upper bound of the running time of an algorithm.

Let's understand what the statement means with the help of a graph.

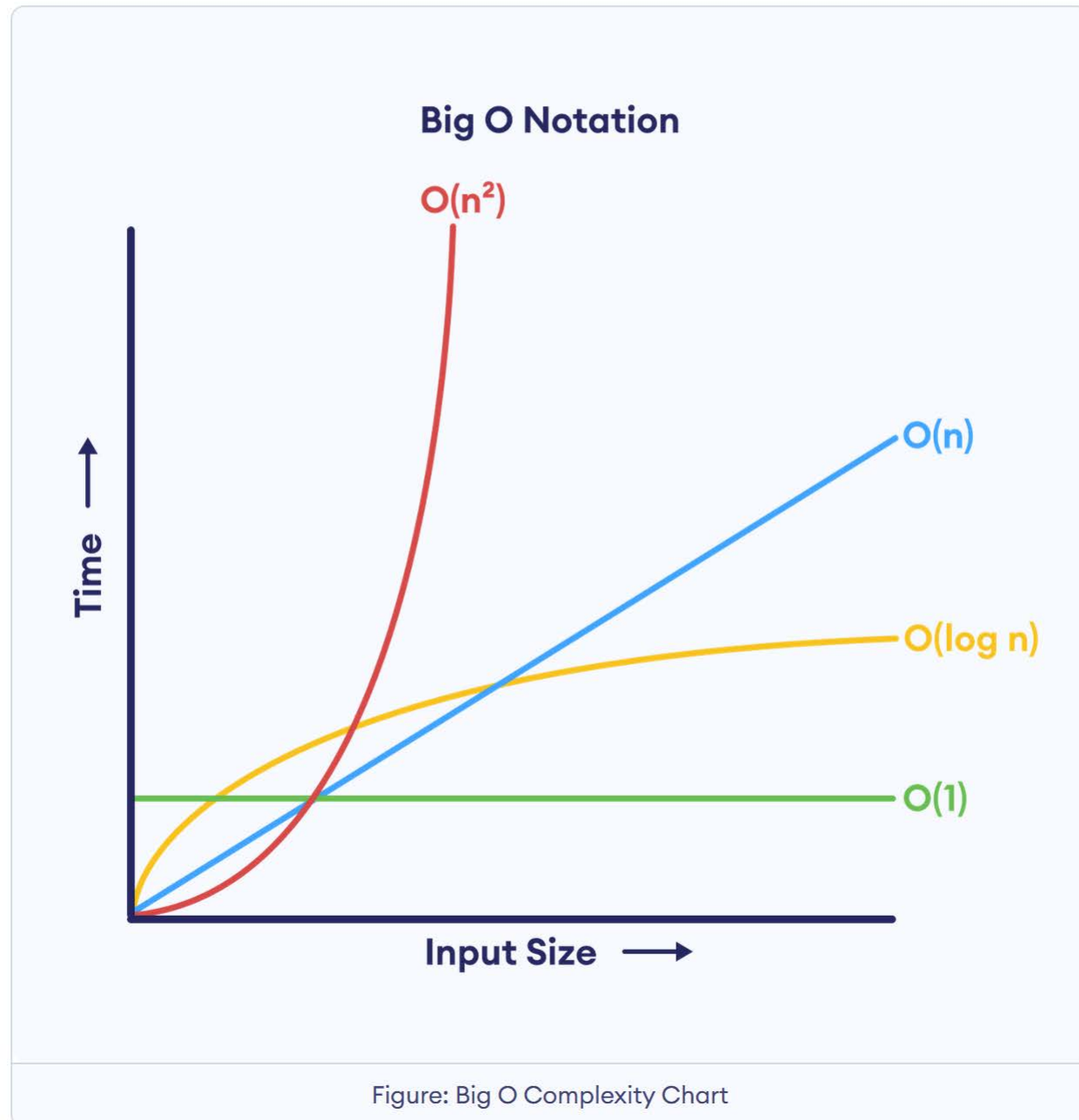


Here, $f(n)$ is the graph of our function.

Introduction to Big O

Let's understand the Big O notation in detail in this lesson.

In computer science, the Big O notation explains how the time complexity grows as the input size increases.



Next, we will learn what the **Big O** notation means mathematically.

Introduction

Asymptotic notations are mathematical notations used to describe the time complexity of an algorithm.

We have already briefly discussed the Big Oh notation (Big O). Now, let's learn about all the commonly used notations to describe time complexity.

There are mainly three asymptotic notations:

- **Big Oh notation (O)** - upper bound
- **Omega notation (Ω)** - lower bound
- **Theta notation (Θ)** - exact bound

Among these three, Big O and Theta notations are particularly useful while analyzing algorithms.

Was this helpful?

Yes 

No 

Quiz:

Time Complexity

What is the time complexity of the following function?

$$f(n) = 6n^5 + 5n^4 + 3n^2 + n$$

A. $O(6n^5)$

B. $O(5)$

C. $O(n^5)$



Correct Answer!

Yes, the time complexity is determined by the term with highest degree $6n^5$ but taken without its coefficient **6**.

D. $O(6)$

Was this helpful?

Yes

No

Quiz:

Time Complexity

What is the time complexity of the following function?

$$f(n) = 6n^5 + 5n^4 + 3n^2 + n$$

A. $O(6n^5)$

B. $O(5)$

C. $O(n^5)$

D. $O(6)$

Was this helpful?

Yes 

No 

Calculating Time Complexity

The time complexity of a function is the term with the highest degree but taken without its coefficient (a constant that multiplies the variable). For example,

The function of the steps required to complete the sum of the first n natural numbers was:

$$f(n) = 2n + 2$$

Here, the term with the highest degree is $2n$. Since we don't include the coefficient in time complexity, its time complexity is n .



Note: We generally denote time complexity as $O(n)$ instead of just n . We will discuss the notation for time complexity in the next chapter.

This means our program takes n steps to complete.

Example

Let's take another example.

$$f(n) = 5n^3 + 2n^2 + 10n$$

Here, the degree of the polynomial is **3**. The term with the highest degree is $5n^3$. Hence, its time complexity is $O(n^3)$.

Thus, a program with $O(n^3)$ time complexity takes n^3 steps to complete.

Was this helpful?

Yes

No

the degree is 1


$$2n^1 + 2n^0$$

Figure: Degree of a Polynomial

Example

Let's take another example.

$$f(n) = 5n^3 + 2n^2 + 10n$$

Its degree is:

the degree is 3

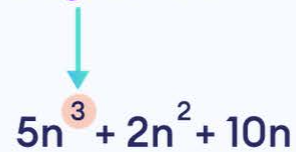

$$5n^3 + 2n^2 + 10n$$

Figure: Degree of a Polynomial

Next, we will learn to calculate the time complexity of an equation.

Was this helpful?

Yes 

No 

Degree of a Polynomial

The degree of a polynomial is the largest exponent of an equation. For example,

$$f(n) = 2n + 2$$

Here, this polynomial has two terms: **2n** and **2**.

To find the degree of this polynomial, we can rewrite it as

$$f(n) = 2n^1 + 2n^0$$

As you can see, the largest exponent in the polynomial is **1**. Thus, the degree of this polynomial is also **1**.

the degree is 1


$$2n^1 + 2n^0$$

Figure: Degree of a Polynomial

Example

Let's take another example.

$$f(n) = 5n^3 + 2n^2 + 10n$$

Analysis of Algorithms

There are three ways to analyze an algorithm:

- **Best Case** - The scenario where the algorithm performs at its best.
- **Average Case** - The scenario where the algorithm performs on average.
- **Worst Case** - The scenario where the algorithm performs at its worst.

Consider the following list/array:

```
numbers = [9, 11, 12, 18, 15]
```

Best case

Suppose we have to search **9** from the `numbers` list.

Then, we would find **9** in the first search. If the size of the list increases, we would still be able to search **9** in our first search. This is the **best-case** analysis.

Worst case

Suppose we have to search **19** from the `numbers` list.

Then, it would take us **5** steps to search **19**. It's because the list doesn't contain **19**, and we have to search until the end of the list to check if **19** is present in the list or not.

This is the **worst-case** analysis because we have to go through every element present in the list.

Average case

We might need to search for an element that is somewhere in between the first and last elements.